Bespokible: Technical Design & Process Document

Overview

This document defines the architecture, user roles, permissions, and onboarding flow for Bespokible, a SaaS product intended for multi-tenant usage. The platform has three user levels and a brand-organization structure that allows granular role-based access and configurations.

1. User Roles & Hierarchy

1.1 System Admins (Internal Team)

- Access to all organizations and brands

- Can impersonate any client user

- Manage subscriptions, billing, analytics, feature toggles, etc.

1.2 Client Admins

- Own the organization (client)

- Can:

- Create/manage brands

- Invite team members to brands

- Configure brand-level settings

- Manage billing

1.3 Client Team Members

- Assigned to specific brands

- Can be assigned one of the following roles:

- Supervisor

- POS User

- Accounts Manager

- E-commerce Manager

- Role determines access and permissions at the brand level

2. Core Entities & Relationships

2.1 Organization (Client)

- Fields:

- id, name, created_by, subscription_tier, status, created_at, updated_at

2.2 Brand

- Belongs to one Organization

- Fields:

- id

- client_id (FK to Organization)

- name

- settings (JSON)

- created_by

- created_at, updated_at

2.3 User

- Global identity for login/authentication

- Fields:

- id, name, email, password, global_role (SystemAdmin, ClientAdmin, TeamMember), status, created_at, updated_at

2.4 UserRelations (Unified Role Table)

- Maps users to brands and their role/permissions

- Fields:

- id

- user_id (FK to User)

- client_id (FK to Organization)

- brand_id (FK to Brand) — nullable for client admins

- brand_role (e.g., Floor Manager, Supervisor)

- permissions (JSON or bitmask/enum)

- status, created_at, updated_at

2.5 Impersonation Logs (Optional)

- Tracks impersonation activity for audit purposes

- Fields:

- system_admin_id (FK to User)

- impersonated_user_id (FK to User)

- started_at, ended_at

3. Onboarding Flow

3.1 Client Sign-Up

1. Client Admin signs up

2. Organization is created

3. Email verified or manually approved

4. Redirected to brand creation

3.2 Create Brand

1. Set brand name/logo

2. Configure basic settings

3. Invite team members to brand with roles

4. Redirect to brand dashboard

4. Brand Roles & Permissions

Available Roles:

- Floor Manager: Oversee daily operations, assign/track tasks

- Supervisor: Monitor performance, limited team control

- POS User: Handle customer sales, view limited order data

- Accounts Manager: View and manage payments, invoices, and revenue

- E-commerce Manager: Manage online listings, track fulfillment

Each role has specific permission sets, configurable per brand if needed (via permissions field).

5. System Admin Impersonation

- Secure mode to impersonate any client user

- Impersonation must be explicitly initiated and logged

- Cannot perform restricted actions unless explicitly allowed

- Log includes:

- system_admin_id, impersonated_user_id, started_at, ended_at

6. Tech Recommendations

Backend: CodeIgniter (CI4)

- Framework: CodeIgniter 4 (modular, REST API ready)

- Auth: JWT-based or session-based auth via filters/middleware

- Routing: RESTful controllers grouped by functionality (auth, users, brands, etc.)

- RBAC: Custom middleware or policy logic

- Models:

- Users, Organizations, Brands, UserRelations, ImpersonationLogs

Frontend: Next.js

- Framework: React-based SSR/SSG

- Auth: JWT stored in HTTP-only cookies

- API Access: fetch() or axios to CodeIgniter endpoints

- Routing:

- /login, /dashboard, /brands/[id], /impersonate/[userId]

- Role-Based UI: Conditional rendering of views and menus

- State Management: SWR or React Query for API data

6.1 Authentication Flow

JWT-Based Auth Flow:

1. User logs in via POST /api/login (CI4)

2. JWT is returned with:

- user_id, global_role, brand_roles, organization_id

3. Frontend stores JWT in HTTP-only cookie

4. Protected Next.js pages fetch /api/me to hydrate session

5. Route guards/middleware restrict access based on roles

7. Future Considerations

- Support for brand-level custom domains (CNAME)

- Brand-specific reports and dashboards

- Multi-org user access (single login across orgs)

- Activity logs per brand/user

- Usage-based billing models

- Public APIs and webhooks per brand

8. Developer Task Breakdown

Phase 1: Core Models & Auth

- Migrate & seed: users, organizations, brands, user_relations

- JWT-based auth endpoints

- Email verification + forgot/reset password

Phase 2: Brand & Team Management

- Brand creation APIs and screens

- User invitation and role assignment

- Brand/team switcher (Next.js UI)

Phase 3: Admin Tools & Impersonation

- System admin dashboard

- Impersonation logic + UI alerts

- Audit logs, filters, and restrictions

This document forms the implementation blueprint for Bespokible, and supports long-term scalability, security, and multi-tenant functionality. All the above table structures are suggestive only, thoughtful logic should be applied while creating table structures keeping performance and scalability in mind.

All of the rest tables required in this project should include *client_id*, *brand_id* or *user_id* with proper FK and cascading logic in place.

Git Operations

- **master** branch will be protected and sync to production
    - Allowed to merge: **developers via MRs**
- **staging** branch will sync to dev site for final checking before production
    - Allowed to push: **developers**
    - Allowed to merge: **developers via MRs**
- create and push **descriptive branch** names like…
    - **feature/login-ui** – For a feature branch for login UI
    - **bugfix/header** – For a branch fixing a header bug
    - **hotfix/cache** – For a critical hotfix for cache
    - **release/v1.2.0** – For a release branch for version 1.2.0